



P00

Diagramas de Estados (UML)

Prof. Edson José Pacheco

2026

Fundamentos de Máquinas de Estados

- ▶ Compreender o ciclo de vida de objetos
- ▶ Identificar o uso de diagramas de estados
- ▶ Distinguir estados, eventos e transições
- ▶ Interpretar a notação visual UML

Elementos do Diagrama

- ▶ Modelar estados simples e compostos
- ▶ Aplicar guardas em transições
- ▶ Definir comportamentos (entry, exit, do)
- ▶ Transição interna vs. Auto-transição

Modelagem Avançada

- ▶ Estados compostos com sub-máquinas
- ▶ Trabalhar com diferentes tipos de eventos
- ▶ Construir diagramas de estados completos
- ▶ Converter diagramas em tabelas

Implementação Prática

- ▶ Implementar o Padrão State em Python
- ▶ Traduzir UML em código orientado a objetos
- ▶ Eliminar condicionais complexos (if/else)
- ▶ Aplicar boas práticas de modelagem



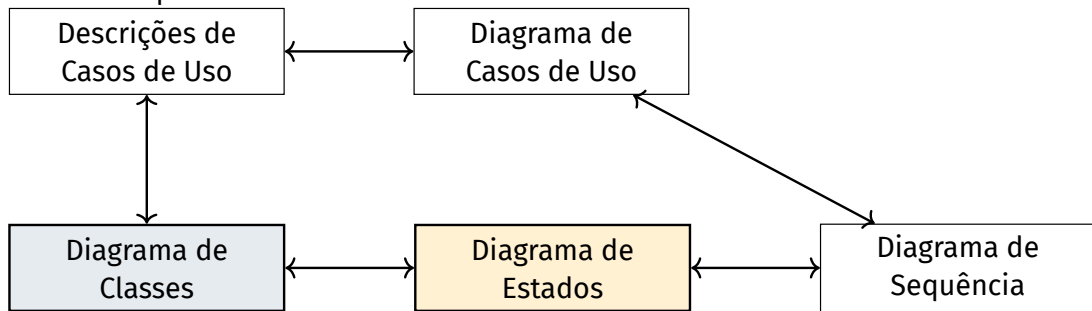
CLIQUE PARA ABRIR NO NAVEGADOR

 Um *Overview* sobre Diagrama de Estados

Todo objeto possui um tempo de vida definido: nasce na criação e deixa de existir na destruição. Durante esse intervalo, o objeto atua enviando e respondendo mensagens.

- ▶ **Máquina de Estados:** Especifica as sequências de estados pelos quais um objeto passa em resposta a eventos.
- ▶ **Aplicação:** Essencial quando o comportamento atual depende do histórico passado.
- ▶ **Foco:** Modelagem do comportamento dinâmico de um objeto individual.

O Diagrama de Estados situa-se na **Especificação de Análise**, relacionando-se com classes e sequências.

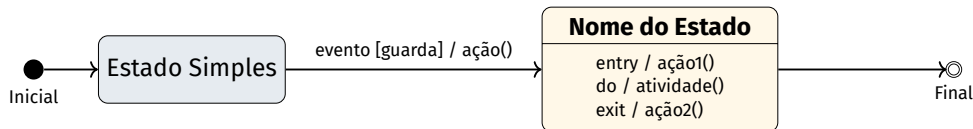


Vértices e Estados

- ▶ **Estado:** Condição estável durante um período no ciclo de vida.
- ▶ **Pseudo-estado:** Vértices transitórios (Início, Término, Escolha).
- ▶ **Ação vs Atividade:** Ações são atômicas; atividades levam tempo.

Dinâmica

- ▶ **Transição:** Mudança entre estados, ativada por eventos.
- ▶ **Evento:** Estímulo que dispara uma transição.
- ▶ **Guarda:** Condição booleana que habilita a transição.



Sintaxe de transição: evento(params) [guarda] / ação(params)

Eventos Explícitos

- ▶ **Sinal:** Comunicação assíncrona.
Ex: `botaoPressionado`
- ▶ **Chamada:** Invocação síncrona.
Ex: `depositar(valor)`
- ▶ **Tempo:** Decurso de tempo.
Ex: `after(5s)`, `at(12:00)`

Eventos Implícitos

- ▶ **Mudança:** Condição verdadeira.
Ex: `when(saldo < 0)`
- ▶ **Completion:** Automático quando do/termina.
- ▶ **Any:** Qualquer evento não tratado.

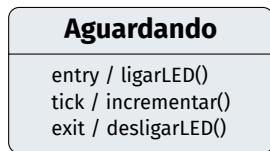
Estados podem definir comportamentos internos:

- ▶ **entry** / Executado ao entrar no estado (atômico).
- ▶ **exit** / Executado antes de sair do estado.
- ▶ **do** / Atividade contínua (pode ser interrompida).
- ▶ **evento / ação** Transição interna: não executa entry/exit.

Processando

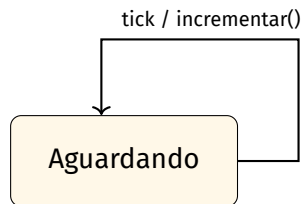
entry / notificar()
do / validar()
consulta / status()
exit / log()

Transição Interna



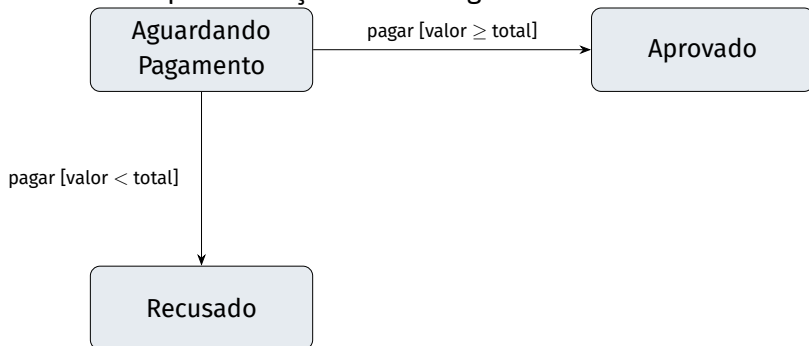
Evento tick: executa incrementar()
NÃO executa entry/exit

Auto-Transição



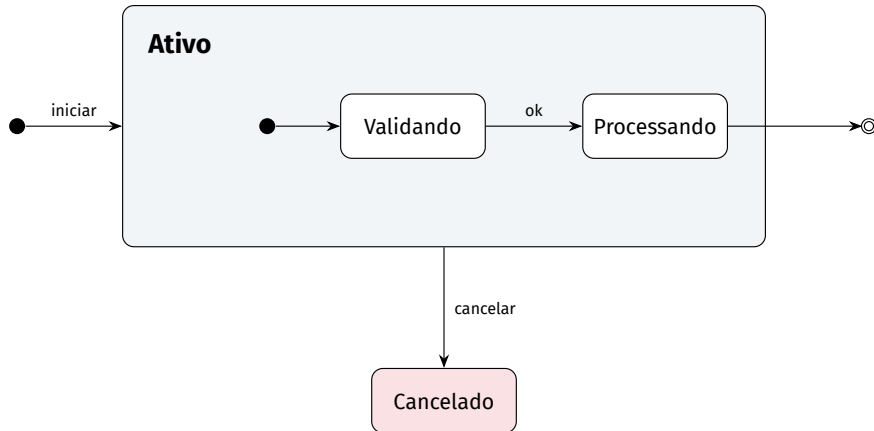
Evento tick: sai e entra no estado
EXECUTA exit → ação → entry

As **guardas** consistem em condições booleanas avaliadas **após** a ocorrência de um evento para determinar qual transição será deflagrada.

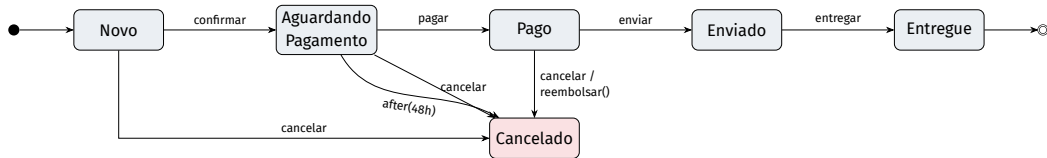


- ▶ Transições originadas no mesmo vértice e ativadas pelo mesmo evento devem possuir guardas **mutuamente exclusivas** para garantir o determinismo.
- ▶ Caso nenhuma condição de guarda seja satisfeita, a transição não é deflagrada e o evento é descartado.

Um estado pode conter uma **sub-máquina** completa.



Transição “cancelar” pode ocorrer de **qualquer** sub-estado de “Ativo”.

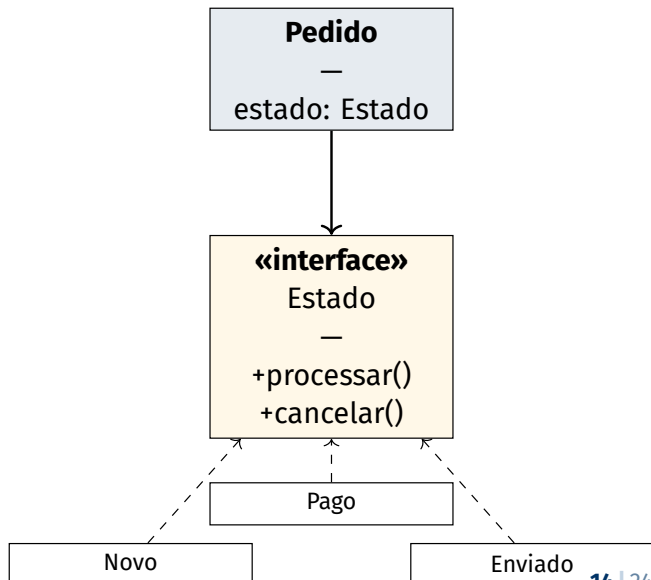


Estrutura

- ▶ Cada estado é uma **classe concreta**
- ▶ Transições são **métodos** que alteram o estado
- ▶ Contexto delega ao estado atual
- ▶ Elimina condicionais extensos

Benefícios

- ▶ Código organizado e extensível
- ▶ Facilita testes unitários
- ▶ Novos estados não afetam existentes



Modelagem

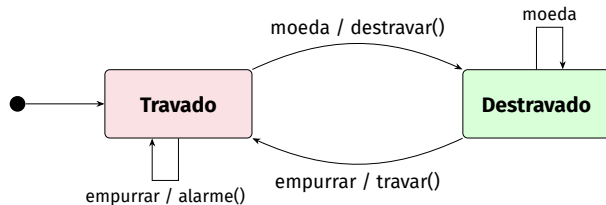
- ▶ Nomeie estados com **substantivos** ou **particípios** (Aguardando, Ativo)
- ▶ Nomeie eventos com **verbos** (conectar, cancelar)
- ▶ Sempre inclua estado inicial
- ▶ Evite estados genéricos sem semântica clara

Implementação

- ▶ Use o **Padrão State** para objetos complexos
- ▶ Para máquinas simples, enum + match pode bastar
- ▶ Implemente **logs** em entry/exit
- ▶ Trate eventos inválidos explicitamente

Exemplo





Estados (2):

- ▶ **Travado:** Bloqueado, não permite passagem
- ▶ **Destravado:** Liberado para uma passagem

Eventos (2):

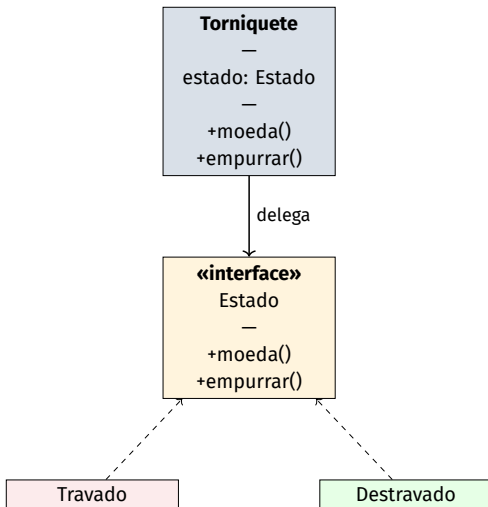
- ▶ **moeda:** Usuário insere moeda
- ▶ **empurrar:** Usuário empurra a catraca

Comportamento:

- ▶ Travado + moeda → Destrava
- ▶ Destravado + empurrar → Trava
- ▶ Travado + empurrar → Alarme!
- ▶ Destravado + moeda → (ignora)

Estado Atual	Evento	Próximo Estado	Ação
Travado	moeda	Destravado	destravar()
Travado	empurrar	Travado	alarme()
Destravado	moeda	Destravado	-
Destravado	empurrar	Travado	travar()

- ▶ Cada linha representa uma **transição possível**
- ▶ A tabela é equivalente ao diagrama de estados
- ▶ Útil para implementação e testes

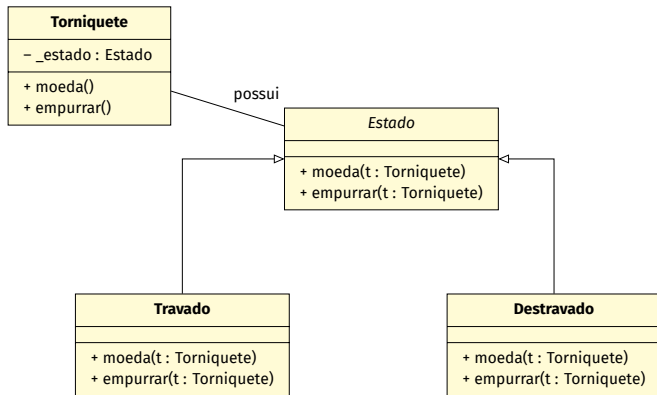


Estrutura:

- ▶ **Contexto:** Mantém referência ao estado atual
- ▶ **Estado:** Interface com os eventos
- ▶ **Estados Concretos:** Implementam comportamento específico

Vantagens:

- ▶ Elimina if/switch extensos
- ▶ Cada estado isolado em sua classe
- ▶ Fácil adicionar novos estados



Estrutura:

- ▶ **Contexto:** Mantém referência ao estado atual
- ▶ **Estado:** Interface com os eventos
- ▶ **Concretos:** Implementam comportamento

Vantagens:

- ▶ Elimina if/switch extensos
- ▶ Cada estado isolado em sua classe
- ▶ Fácil adicionar novos estados

```
class Estado(ABC):  
    """Interface para os estados do torniquete."""  
  
    @abstractmethod  
    def moeda(self, torniquete: 'Torniquete') -> None:  
        """Evento: usuário insere moeda."""  
        pass  
  
    @abstractmethod  
    def empurrar(self, torniquete: 'Torniquete') -> None:  
        """Evento: usuário empurra a catraca."""  
        pass  
  
    def __str__(self) -> str:  
        return self.__class__.__name__
```

```
class Travado(Estado):
    def moeda(self, torniquete: 'Torniquete') ->
        None:
            print(" Moeda aceita!")
            print(" Destravando...")
            torniquete._estado = Destravado()

    def empurrar(self, torniquete: 'Torniquete') ->
        None:
            print(" Bloqueado! Insira uma moeda.")
            print(" ALARME!")

class Destravado(Estado):
    def moeda(self, torniquete: 'Torniquete') ->
        None:
            print(" Moeda extra ignorada (já destravado
            ).")

    def empurrar(self, torniquete: 'Torniquete') ->
        None:
            print(" Passagem liberada!")
            print(" Travando...")
            torniquete._estado = Travado()
```

```
class Torniquete:
    def __init__(self):
        self._estado: Estado = Travado()
        print("=" * 40)
        print("  TORNIQUETE INICIADO")
        print("=" * 40)

    @property
    def estado(self) -> str:
        return str(self._estado)

    def moeda(self) -> None:
        print(f"\n[Estado: {self._estado}] -> Evento:
        MOEDA")
        self._estado.moeda(self)
        print(f"[Novo estado: {self._estado}]")

    def empurrar(self) -> None:
        print(f"\n[Estado: {self._estado}] -> Evento:
        EMPURRAR")
        self._estado.empurrar(self)
        print(f"[Novo estado: {self._estado}]")
```



► Download .py

Dominando Diagramas de Estados e o Padrão State

Fundamentos e Notação UML



O que é um Estado?

Condição estável de um objeto durante um período do seu ciclo de vida.

Dinâmica das Transições



'event'

disparadas por eventos



guard

validadas por guardas



ação

que podem executar ações



Comportamentos Internos

Ações automáticas executadas na entrada (entry) durante (do) o estado

saída (exit) do estado.

Este guia resume a modelagem do comportamento dinâmico de objetos através de Máquinas de Estados. Ele aborda desde a notação visual de UML até a tradução prática para código orientado a objetos, eliminando condicionais complexas.

empurrar

alarme()



moeda

destravar()



Travado

Destravado

empurrar

alarme()



empurrar

travar()

Exemplo Prático:
Torniquete (Catraca)

Implementação com Padrão State

Estrutura do Padrão State

Cada estado torna-se uma classe concreta que implementa comportamentos específicos do objeto.



State A



State B



State C

Fim dos "If/Else" Complexos

O contexto delega decisões ao estado atual, tornando o código limpo e extensível.



Boas Práticas de Nomenclatura

Aguardando

substantivo

estados

pagar, cancelar

verbos

eventos

Use substantivos para estados (Aguardando) e verbos para eventos (pagar, cancelar).



Prof. Edson José Pacheco

pacheco@ufpr.br

UFPR

UNIVERSIDADE FEDERAL DO PARANÁ